

Doc. No.: P1256R0

Date: 2018-10-07

Reply to: Detlef Vollmann, dv@vollmann.ch

Audience: SG1, LEWG, WG21

Executors Should Go To A TS

Abstract

Many people inside and outside WG21 would like to have executors in C++20. I'm one of them.

However, this paper argues that executors are way too important and usage experience is still too low to put executors forward now for C++20. However it's very important to get executors out now as a TS.

Introduction

The executors proposal tries to provide a common interface to all kind of task execution mechanisms. The goal is to have an interface that allows a single implementation of a potentially concurrent control structure. This single implementation should still run on very different implementations of the executor interface.

Such an approach is much needed especially to ease the usage of other execution mechanisms than the well known OS threads.

This is quite novel and needs at least some usage experience from all kind of concurrent control structures with all kinds of executors before it can be standardized.

History

The first executors proposals that were presented to SG1 were mainly based on coarse-grain concurrency tasks typically run on OS threads.

With N3388 (and follow-ups) another approach was presented that came out of experience with continuation based concurrency based on OS synchronization.

N4406 introduced the requirements on executors from a parallelism perspective.

P1055 again is based on continuation, but allows for more control over the actual execution of these continuations.

Missing experience

P0443 grew out of a small group who made sure that the requirements of their use cases were covered by the proposal. That small group was pretty effective to produce a proposal that could get consensus inside SG1.

But while SG1 is a group with a lot of expertise in parallelism and concurrency, it's still a small group and can't reflect all kind of use cases.

So when somebody outside that group tried to take the proposal and implement their own use case, they promptly came up with problems in the proposal (which resulted in P1055). So now the new proposal covers one

use case more, but still doesn't have wide usage experience.

At the meeting in Oulu in 2016, I presented some challenges from a concurrency perspective that an executor proposal would need to solve. In P1257 (which will be a late paper for San Diego and not part of the pre San Diego mailing, it will be attached to the SG1 wiki for San Diego) I used the proposal in P0443 and tried to implement one of the examples in the presentation from Oulu. I promptly came up with new problems that P0443 doesn't solve.

The consequence of this should not be to delay the executors proposal another round and incorporate this specific use case as well, but to put out what we have as a TS to collect as many use cases as possible, so that the final result is not only based on a small group of geniuses, but also on the feedback of a wide community.

Dependencies

Networking

SG1 has decided that the Networking TS should not be merged into the C++ working paper before executors go in. This is based on the fact that the Networking TS also contains an executor model and it's important that networking uses the executor model that's upcoming from SG1.

While that's completely true, P1259 proposes to merge those parts of the Networking TS that don't depend on executors. The major part of the Networking TS is not dependent on executors!

.then

The Concurrency TS 1 contains extensions to `std::future`. These extensions are really part of the executor model. `.then()` has seen quite some usage experience in the isolated world of `std::future`. This experience has surfaced several problems of the specification in Concurrency TS 1 which are addressed by the executor model. For this reason `.then()` and other extensions to `std::future()` need to wait until executors are ready.

Conclusion

The executor proposal is not something that pre-existed as library and got wide usage experience. To get this experience, we need to publish the proposal as TS. Putting it into the C++ working paper would limit the proposal to the use cases of a very small group.

Final words

Several people have worked very hard to get the executors proposals to the stage where it currently is. This is very much appreciated and my thanks go to all who participated in this work. But given all the open questions that still exist, probably even more work is needed to make executors ready for C++23.

And sorry, this paper was written too late and didn't have time to be reviewed by the authors of the proposals mentioned. So I may have misinterpreted their background.